



Deep Learning

Logistic Regression Classifier

Dr. Mehran Safayani



safayani@iut.ac.ir



safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/deep_learning_course



Supervised Learning

Input(x)	Output (y)	Application
Home features	Price	Real Estate
Ad, user info	Click on ad? (0/1)	Online Advertising
Image	Object (1,...,1000)	Photo tagging
Audio	Text transcript	Speech recognition
English	Chinese	Machine translation
Image, Radar info	Position of other cars	Autonomous driving

Basics of Neural Network Programming

Binary Classification

Binary Classification



—————→ 1 (cat) vs 0 (non cat)

		Blue			
Green		255	134	93	22
Red		255	134	202	22
		255	231	42	22
		4	30		
		123	94	83	2
		192	124		
		34	44	187	92
		34	76	232	124
		94			
		67	83	194	202

Binary classification

			Blue					
		Green		255	134	93	22	
	Red		255	134	202	22	2	
		255	231	42	22	4	30	
		123	94	83	2	192	124	
		34	44	187	92	34	142	
		34	76	232	124	94		
		67	83	194	202			

$$\vec{x} = \begin{bmatrix} 255 \\ 231 \\ \vdots \\ 254 \\ 253 \\ 250 \\ 220 \end{bmatrix} \quad 64 \times 64 \times 3 = \underbrace{12288}_{n_x}$$

$$\vec{x} \rightarrow \boxed{\text{model}} \rightarrow \hat{y}$$

- Notation

$$(\vec{x}, y) \quad x \in R^{n_x}, y \in \{0,1\}$$

Binary classification

- m training example: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

• $X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$

n_x

m

$$X \in \mathbb{R}^{n_x \times m}$$

$$X.\text{shape} = (n_x, m)$$

$$Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)}]$$

$$Y \in \mathbb{R}^{1 \times m}$$

$$Y.\text{shape} = (1, m)$$

Logistic Regression

- Given $\mathbf{x}$, output $\hat{y} = P(y=1|\mathbf{x})$ $0 \leq \hat{y} \leq 1$

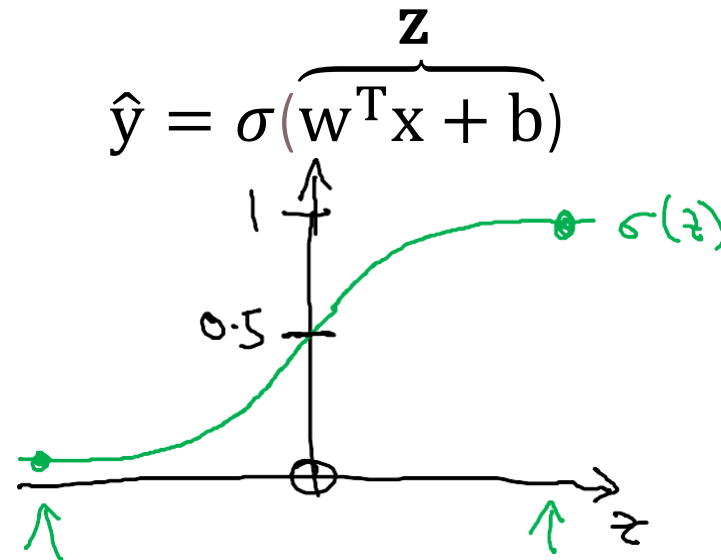
$\mathbf{x} \in \mathbb{R}^{n_x}$ parameters: $\mathbf{w} \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$

if z large $\sigma(z) \approx 1$

if z large negative $\sigma(z) \approx 0$



Logistic Regression

$$\bullet \hat{y} = \sigma(\underbrace{w^T x + b}_z) \quad \hat{y} = w^T x$$

$$x_0 = 1, x \in \mathbb{R}^{n_x+1}$$

$$W = \begin{bmatrix} b = w_0 \\ w_1 \\ w_2 \\ w_3 \\ \vdots \\ \vdots \\ \vdots \\ w_{n_x} \end{bmatrix} \begin{matrix} \left. \vphantom{\begin{bmatrix} b = w_0 \\ w_1 \\ w_2 \\ w_3 \\ \vdots \\ \vdots \\ \vdots \\ w_{n_x} \end{bmatrix}} \right\} b \\ \left. \vphantom{\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ \vdots \\ \vdots \\ w_{n_x} \end{bmatrix}} \right\} w \end{matrix}$$

Logistic Regression cost function

- Loss (error) function: $L(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2 = \frac{1}{2} (\sigma(w^T x + b) - y)^2$ SE: Square Error

- What is the problem?

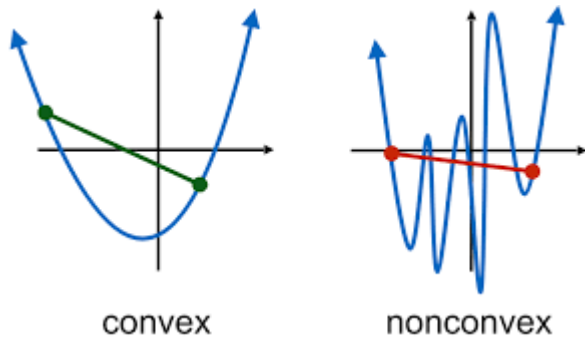
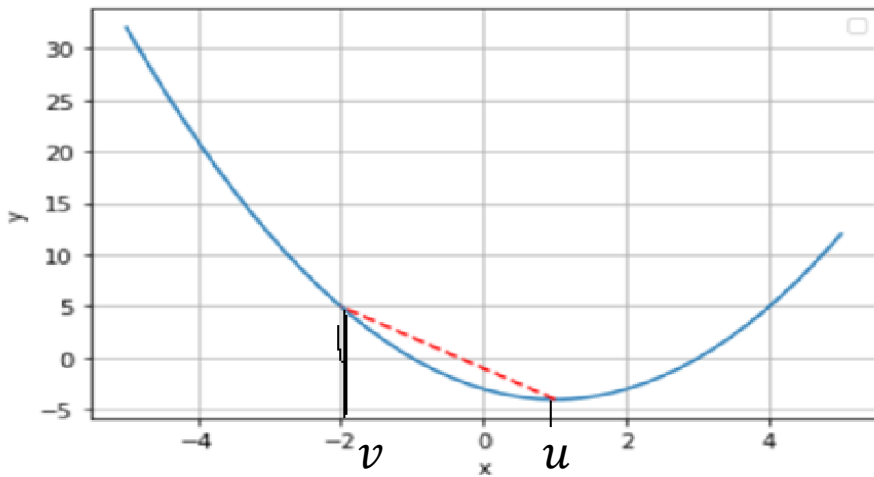
- .

Convexity

Function $h(u)$ with $u \in X$ is **convex** if for any $u, v \in X$ and for any $0 \leq \lambda \leq 1$ we have:

$$h(\lambda u + (1 - \lambda)v) \leq \lambda h(u) + (1 - \lambda) h(v)$$

برای توابع محدب هر بهینه محلی یک بهینه سراسری است.

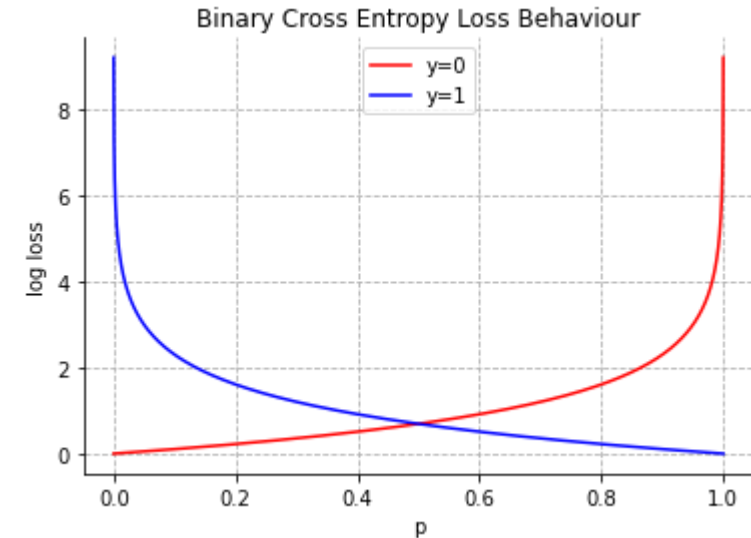


Cross Entropy

- $L(\hat{y}, y) = -(y \log \hat{y} + (1-y) \log(1-\hat{y}))$

if $y=1$: $L(\hat{y}, y) = -\log \hat{y}$

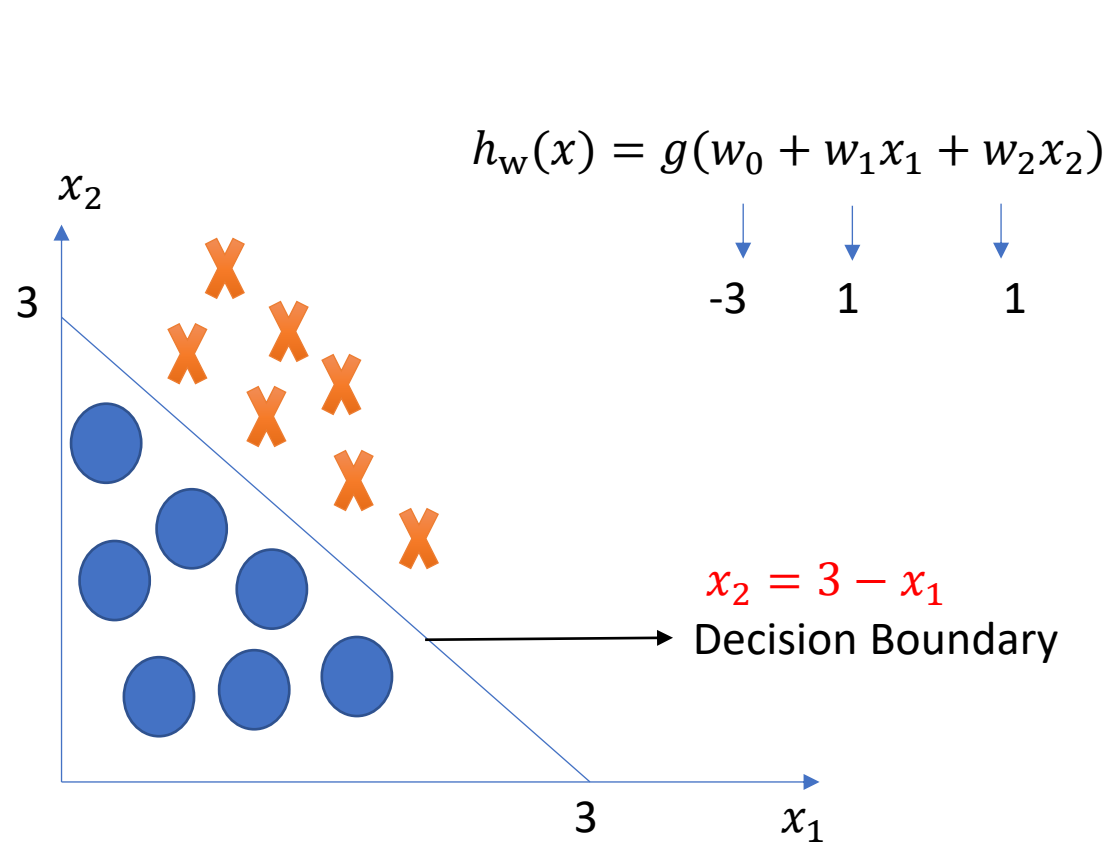
if $y=0$: $L(\hat{y}, y) = -\log(1-\hat{y})$



<https://datamonje.com/classification-loss-functions/>

- Cost function:
$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, \hat{y}^{(i)})$$
$$= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$$

Decision Boundary

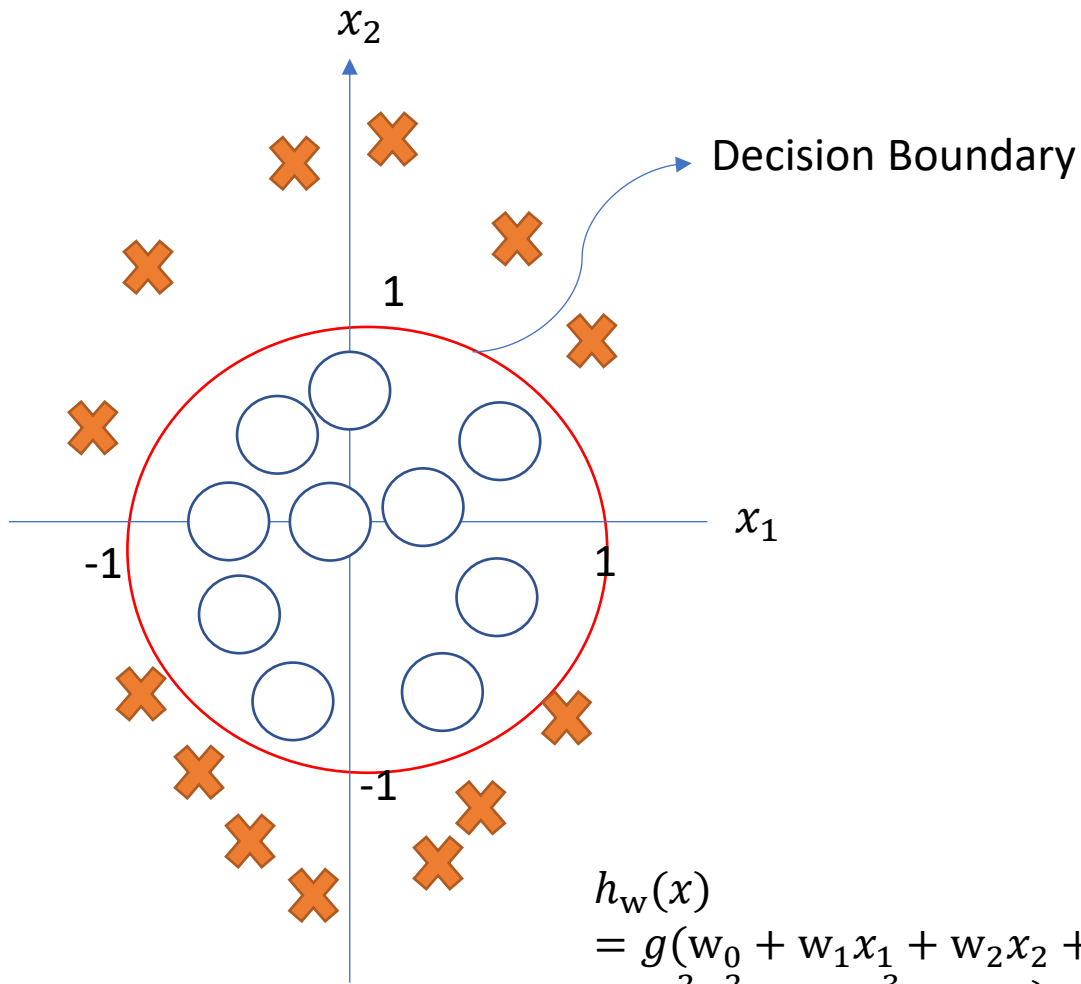


$$w = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict $y = 1, \text{ if } \underbrace{-3 + x_1 + x_2}_{w^T X} \geq 0$

$x_1 + x_2 \geq 3$

Non-Linear Decision Boundaries



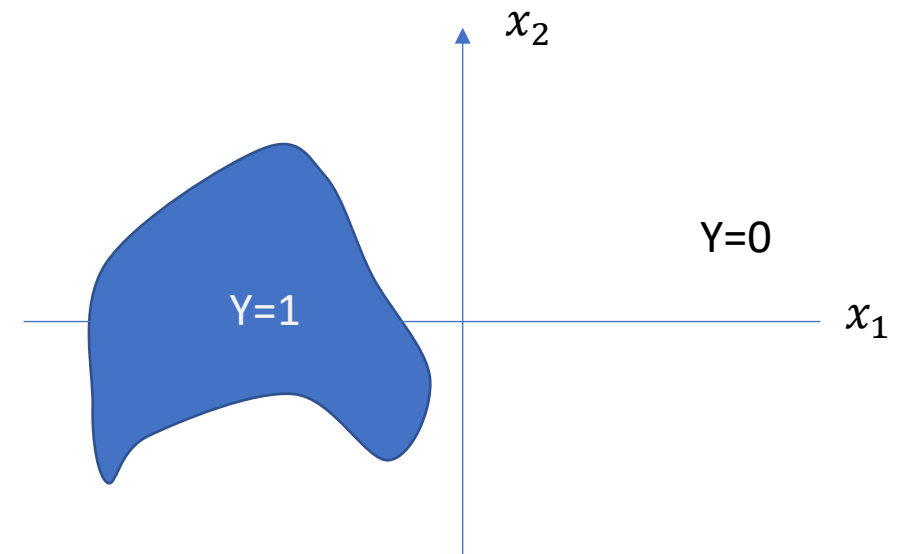
$$h_w(x) = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1^2x_2 + w_5x_1^2x_2^2 + w_6x_1^3x_2 + \dots)$$

$$h_w(x) = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2)$$

$$w = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Predict $y = 1, \text{ if } \underbrace{-1 + x_1^2 + x_2^2}_{w^T X} \geq 0$

$$x_1^2 + x_2^2 \geq 1$$



Gradient Descent

Cost Function

Minimize $J(\mathbf{b}, w_1)$

$\mathbf{b}, w_1$

If $J(w_1) = (w_1 - 2)^2$

$$\frac{dJ(w_1)}{dw_1} = 0$$



$$\frac{dJ(w_1)}{dw_1} = 2(w_1 - 2) = 0$$



$$w_1 = 2$$

Gradient Descent

Minimize $J(\mathbf{b}, w_1)$

$\mathbf{b}, w_1$

Minimize $J(\mathbf{b}, w_1, \dots, w_n)$

$\mathbf{b}, w_1, \dots, w_n$

Repeat until convergence: {

For $j=0, \dots, n$

$b = w_0$

$$w_j = w_j - \alpha \frac{dJ(\mathbf{b}, w_1, \dots, w_n)}{dw_j}$$

}

α is **learning rate**

Updating all w_j *Simultaneously*

Convergence condition:

$$\|W^{t+1} - W^t\|_2 \leq \varepsilon$$

Gradient Descent

Correct form

$$\text{temp0} = \mathbf{b} - \alpha \frac{dJ(\mathbf{b}, \mathbf{w}_1)}{d\mathbf{b}}$$

$$\text{temp1} = \mathbf{w}_1 - \alpha \frac{dJ(\mathbf{b}, \mathbf{w}_1)}{d\mathbf{w}_1}$$

$$\mathbf{b} = \text{temp0}$$

$$\mathbf{w}_1 = \text{temp1}$$



بروزرسانی همزمان

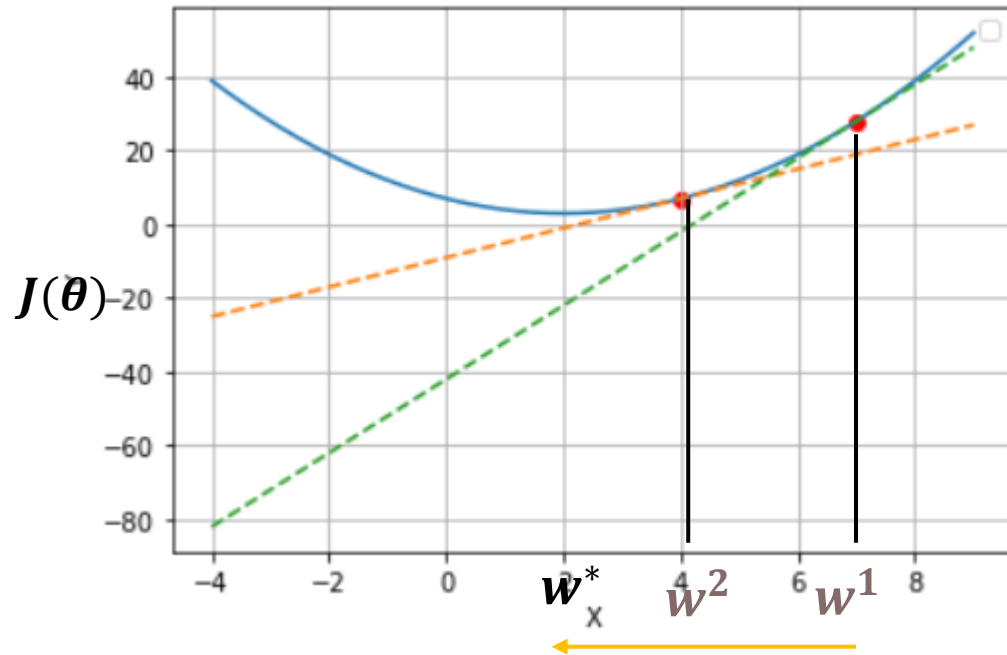
Incorrect form

$$\mathbf{b} = \mathbf{b} - \alpha \frac{dJ(\mathbf{b}, \mathbf{w}_1)}{d\mathbf{b}}$$

$$\mathbf{w}_1 = \mathbf{w}_1 - \alpha \frac{dJ(\mathbf{b}, \mathbf{w}_1)}{d\mathbf{w}_1}$$



Gradient Descent



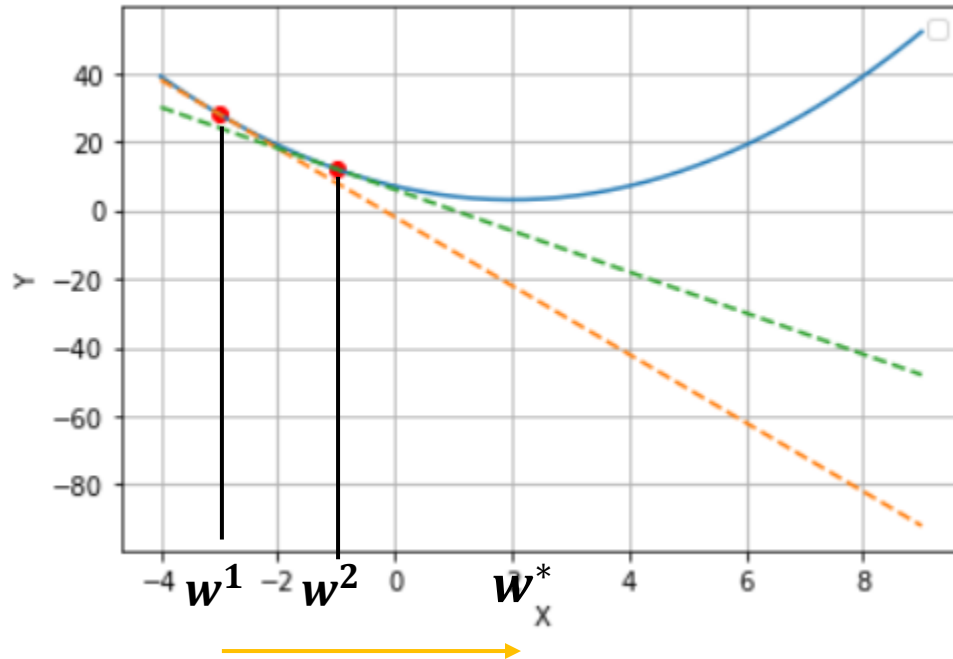
خطوط مماس نشان داده شده دارای شیب یا مشتق مثبت هستند.
در نتیجه:

$$\frac{dJ(w^1)}{dw^1} > 0, \alpha > 0 \Rightarrow \alpha \frac{dJ(w^1)}{dw^1} > 0$$

$$\Rightarrow w^2 = w^1 - \alpha dw^1$$

w کوچکتر میشود و به سمت چپ حرکت میکنیم.

Gradient Descent



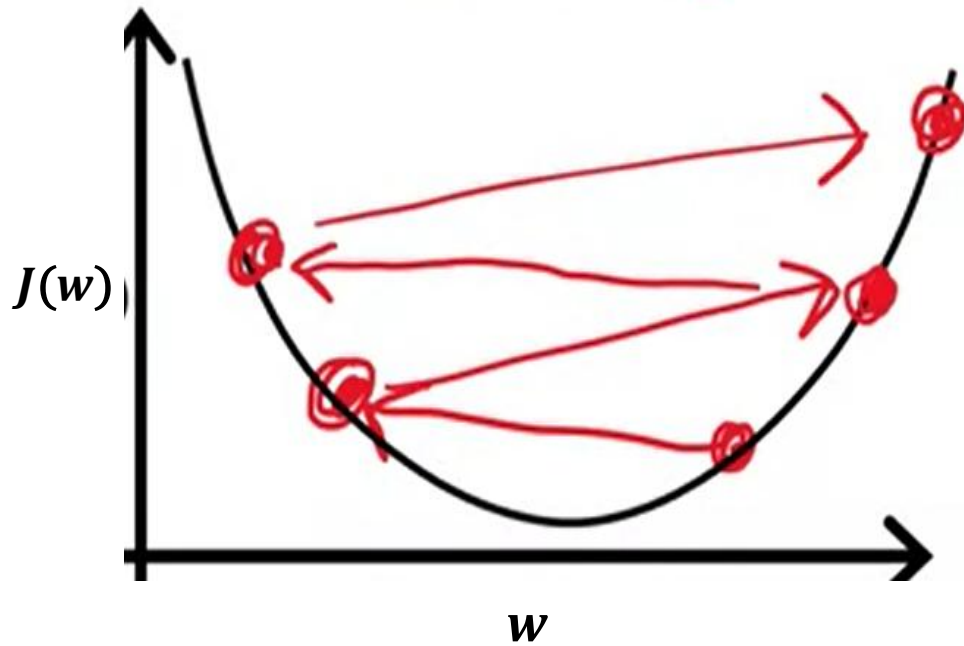
خطوط مماس نشان داده شده دارای شیب یا مشتق مثبت هستند.
در نتیجه:

$$\frac{dJ(w^1)}{dw^1} < 0, \alpha > 0 \Rightarrow \alpha \frac{dJ(w^1)}{dw^1} < 0$$

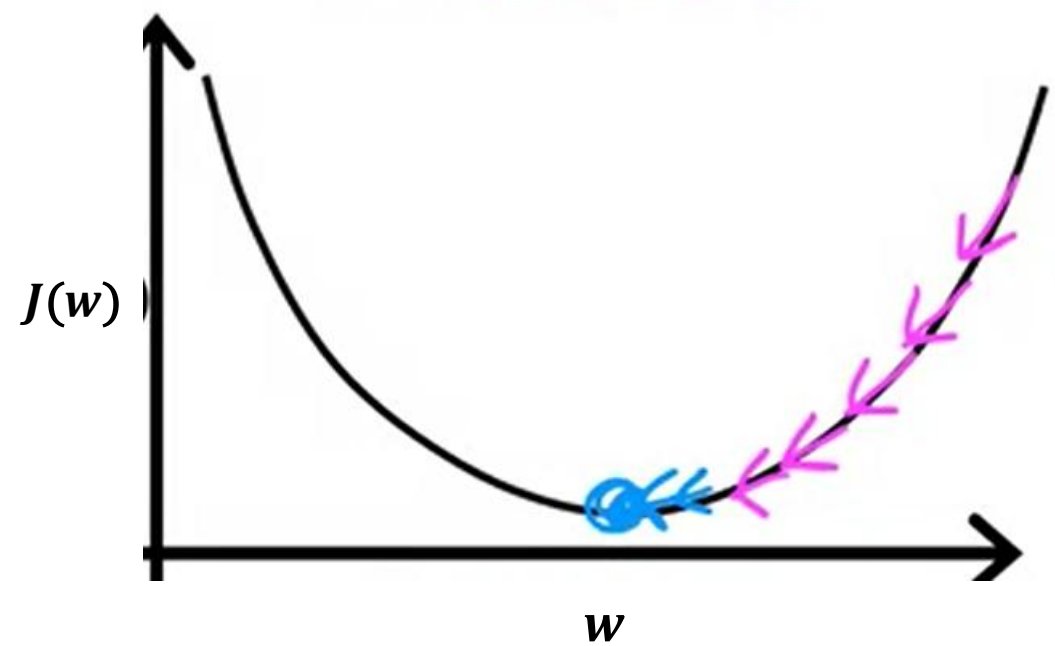
$$\Rightarrow w^2 = w^1 - \alpha \frac{dJ(w^1)}{dw^1}$$

w بزرگتر میشود و به سمت راست حرکت میکنیم.

Choosing Learning Rate

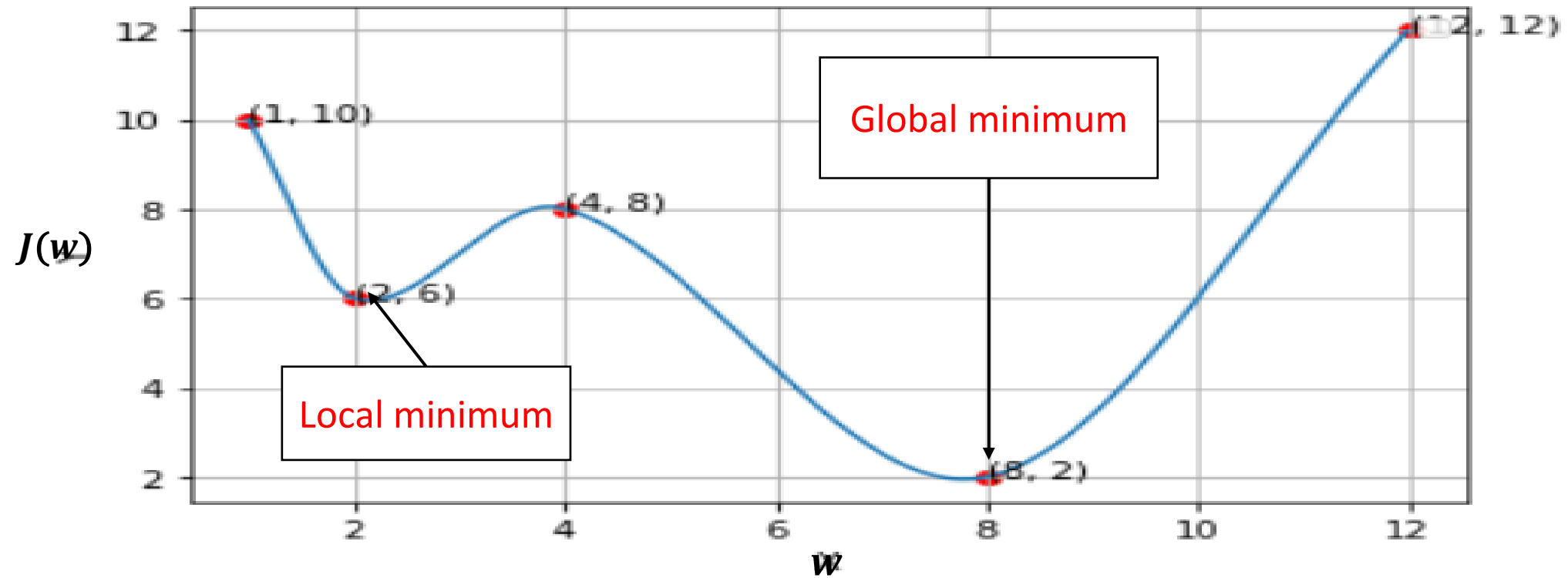


α is too large

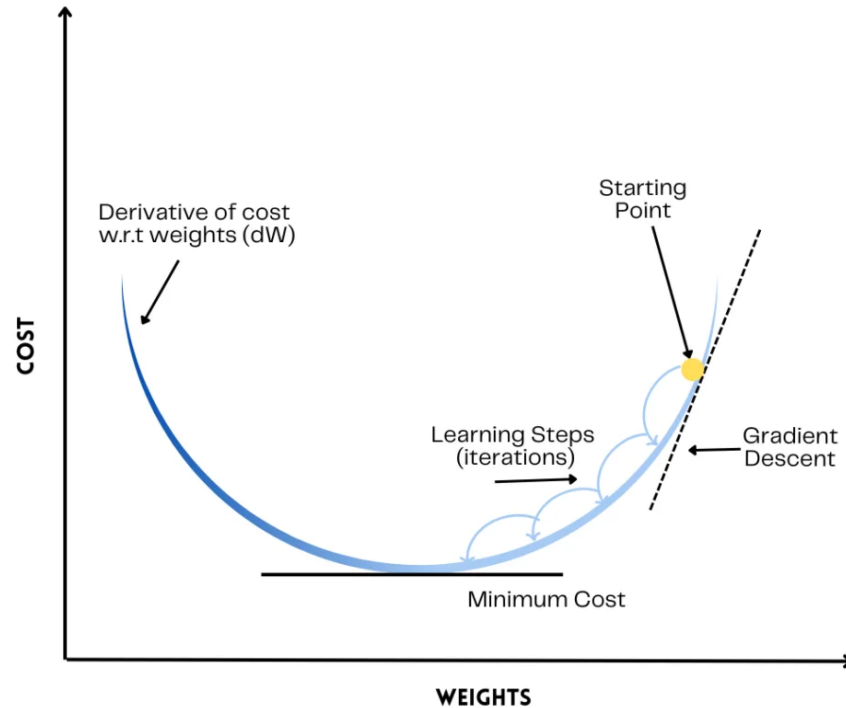


α is small

Gradient Descent Weakness



Gradient Descent



- 1) $\alpha > 0$

Repeat{

$$w = w - \alpha \underbrace{\frac{dJ(w)}{dw}}$$

}until convergence

$$w = w - \alpha dw$$

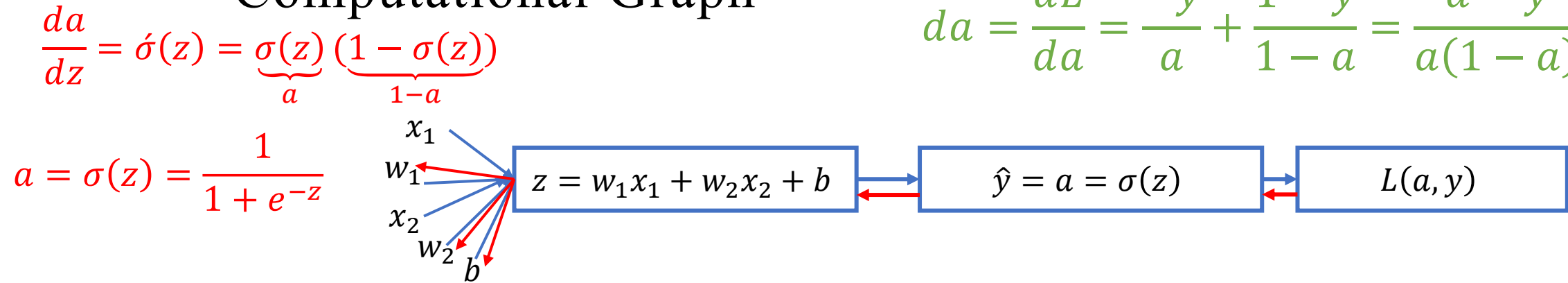
$$z = w^T x + b$$

- $\hat{y} = a = \sigma(z)$
- $L(a, y) = -(y \log a + (1 - y) \log(1 - a))$

Gradient Descent

$$L(a, y) = -(y \log a + (1 - y) \log(1 - a))$$

Computational Graph



$$dz = \frac{dL}{dz} = \frac{dL(a, y)}{dz} = \frac{dL}{da} \times \frac{da}{dz} = \frac{a - y}{a(1 - a)} \times a(1 - a) = a - y$$

$$dw_1 = \frac{dL}{dw_1} = \underbrace{\frac{dL}{dz}}_{dz} \times \underbrace{\frac{dz}{dw_1}}_{x_1} = x_1 dz$$

$$dw_2 = x_2 dz$$

$$db = \frac{dL}{db} = \frac{dL}{dz} \times \underbrace{\frac{dz}{db}}_1 = dz$$

Gradient Descent

- $J(w,b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)}, y^{(i)})$

- $a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(wx^{(i)} + b)$

- $dw_j = \frac{1}{m} \sum_{i=1}^m \frac{dL(a^{(i)}, y^{(i)})}{dw_j}$

Logistic regression on m examples

$J = 0;$ $dw_1 = 0;$ $dw_2 = 0;$ $db = 0;$

$w_1 \leftarrow \text{random}$ $w_2 \leftarrow \text{random}$ $b \leftarrow \text{random}$

Repeat{

For $i=1$ **to** m

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += [y^{(i)} \text{Log} a^{(i)} + (1 - y^{(i)}) \text{Log}(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$$J /= m;$$

$$dw_1 /= m;$$

$$dw_2 /= m;$$

$$db /= m;$$

$$w_1 = w_1 - \alpha dw_1$$

$$w_2 = w_2 - \alpha dw_2$$

$$b = b - \alpha db$$

} until convergence

$$w^t = \begin{bmatrix} w_1^t \\ w_2^t \\ b^t \end{bmatrix} \quad w^{t+1} = \begin{bmatrix} w_1^{t+1} \\ w_2^{t+1} \\ b^{t+1} \end{bmatrix}$$

$$\|w^{t+1} - w^t\|_2 \leq \varepsilon$$

$$dw = \begin{bmatrix} dw_1 \\ dw_2 \\ db \end{bmatrix}$$

$$\|dw\| \leq \varepsilon = 10^{-4}$$

What's wrong with the code?

Logistic regression on m examples

$w_1 \leftarrow \text{random}$ $w_2 \leftarrow \text{random}$ $b \leftarrow \text{random}$

Repeat{

$J = 0;$ $dw_1 = 0;$ $dw_2 = 0;$ $db = 0;$

For $i=1$ to m

$$z^{(i)} = w^T x^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += [y^{(i)} \text{Log} a^{(i)} + (1 - y^{(i)}) \text{Log}(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$$J /= m;$$

$$dw_1 /= m;$$

$$dw_2 /= m;$$

$$db /= m;$$

$$w_1 = w_1 - \alpha dw_1$$

$$w_2 = w_2 - \alpha dw_2$$

$$b = b - \alpha db$$

} until convergence

$$w^t = \begin{bmatrix} w_1^t \\ w_2^t \\ b^t \end{bmatrix} \quad w^{t+1} = \begin{bmatrix} w_1^{t+1} \\ w_2^{t+1} \\ b^{t+1} \end{bmatrix}$$

$$\|w^{t+1} - w^t\|_2 \leq \varepsilon$$

$$dw = \begin{bmatrix} dw_1 \\ dw_2 \\ db \end{bmatrix}$$

$$\|dw\| \leq \varepsilon = 10^{-4}$$

Vectorizing Logistic Regression

- $z=0;$

For i *in* $\text{range}(n_x)$
 $z += w[i] * x[i]$
 $z += b$

} First method

- $z=0;$

$z = \text{np.dot}(w, x) + b$

} Second method
SIMD
GPU

Vectorizing Logistic Regression

$$z^{(1)} = w^T x^{(1)} + b \quad z^{(2)} = w^T x^{(2)} + b$$

$$a^{(1)} = \sigma(z^{(1)}) \quad a^{(2)} = \sigma(z^{(2)})$$

$$z^{(m)} = w^T x^{(m)} + b$$

$$a^{(m)} = \sigma(z^{(m)})$$

$$\bullet X = \begin{bmatrix} | & | & & | \\ \textcolor{brown}{x}^{(1)} & \textcolor{green}{x}^{(2)} & \dots & \textcolor{blue}{x}^{(m)} \\ | & | & & | \end{bmatrix} \in R^{n_x \times m} \quad \underbrace{[w_1 \dots w_{n_x}]}_{W^T} \begin{bmatrix} | & | & & | \\ \textcolor{brown}{x}^{(1)} & \textcolor{green}{x}^{(2)} & \dots & \textcolor{blue}{x}^{(m)} \\ | & | & & | \end{bmatrix}$$

$$\bullet Z = \begin{bmatrix} \underbrace{z^{(1)}}_{w^T x^{(1)} + b} & \underbrace{z^{(2)}}_{w^T x^{(2)} + b} & \dots & \underbrace{z^{(m)}}_{w^T x^{(m)} + b} \end{bmatrix} = w^T X + [b \ b \ \dots \ b]_{1 \times m}$$

$$\bullet Z = \underbrace{np \cdot dot(w \cdot T, X)}_{1 \times m} + \underbrace{b}_{(1,1)} \text{ "broadcasting" } [b, b \dots, b]_{1 \times m}$$

Vectorizing Logistic Regression

- $A = [a^{(1)}, a^{(2)}, \dots, a^{(m)}] = \sigma(\underbrace{Z}_{1 \times m})$
- $dz^{(1)} = a^{(1)} - y^{(1)} \quad dz^{(2)} = a^{(2)} - y^{(2)}$
- $dZ = [dz^{(1)} \ dz^{(2)} \ \dots \ dz^{(m)}]_{1 \times m}$
- $A = [a^{(1)} \ a^{(2)} \ \dots \ a^{(m)}] \quad Y = [y^{(1)} \ y^{(2)} \ \dots \ y^{(m)}]$
- $dZ = A - Y = [a^{(1)} - y^{(1)} \ a^{(2)} - y^{(2)} \ \dots \ a^{(m)} - y^{(m)}]$

Vectorizing Logistic Regression

$$\bullet \begin{bmatrix} dw_1 \\ dw_2 \\ dw_3 \\ \vdots \\ dw_{nx} \end{bmatrix}_{nx \times 1} \left\{ \begin{array}{l} \bullet dw=0 \\ dw += \widetilde{x}^1 \widetilde{dz}^1 \\ dw += x^2 dz^2 \\ \vdots \\ dw += x^m dz^m \\ dw /= m \end{array} \right. \left\{ \begin{array}{l} db=0 \\ db += \widetilde{dz}^1 \\ db += dz^2 \\ \vdots \\ db += dz^m \\ db /= m \end{array} \right.$$

عدد بردار a^{1-y^1}

عدد a^{1-y^1}

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)} = \frac{1}{m} np.sum(dz) \quad 1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ \vdots \\ 1 \end{bmatrix}_{m \times 1}$$

$$db = \frac{1}{m} dz \times 1$$

$$dz = [dz^1 \ dz^2 \ \dots \ dz^m]_{1 \times m}$$

$$dw = \frac{1}{m} [x^{(1)} dz^1 + x^{(2)} dz^2 + \dots + x^{(m)} dz^m]$$

$$\boxed{dw = \frac{1}{m} X dz^T} = \frac{1}{m} \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & \dots & \dots & x^{(m)} \end{bmatrix} \times \begin{bmatrix} dz^1 \\ dz^2 \\ dz^3 \\ \vdots \\ \vdots \\ dz^m \end{bmatrix}$$

Vectorizing Logistic Regression

- $w_1, w_2, b \leftarrow \text{random}$

For iter in range(1000)

1 epoch

$$Z = w^T X + b = np \cdot \text{dot}(w.T, X) + b$$

$$A = \sigma(Z)$$

$$dZ = A - Y$$

$$dw = \frac{1}{m} X dz^T$$

$$db = \frac{1}{m} np.sum(dz)$$

$$w = w - \alpha dw$$

$$b = b - \alpha db$$

$$w = np \cdot \text{random} \cdot \text{randn}(n_x, 1)$$

Logistic Regression Cost function

- $\hat{y} = \sigma(w^T x + b) \quad 0 < \sigma(z) = \frac{1}{1+e^{-z}} < 1$
- $\hat{y} = p(y = 1|x)$
- $\left. \begin{array}{l} \text{if } y = 1 : p(y|x) = \hat{y} \\ \text{if } y = 0 : p(y|x) = 1 - \hat{y} \end{array} \right\} p(y|x)$
- $\boxed{p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}}$ *distribution? Bernoulli*
- $\text{if } y = 1 : p(y|x) = \hat{y}$
- $\text{if } y = 0 : p(y|x) = 1 - \hat{y}$
- $\log p(y|x) = \log[\hat{y}^y (1 - \hat{y})^{1-y}] = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$
 $= -L(\hat{y}, y) \text{ Max Likelihood}$

Logistic Regression Cost function

- $\log P(\text{labels in training set}) = \log \prod_{i=1}^m P(y^i | x^i)$
- $\log P(\cdots) = \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}) = -\sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$
- Cost function
$$\underbrace{J(w, b)}_{\text{minimize}} = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Neural Networks

